

Universidad de Oriente	
Ciencia de datos	Clase 08 - Ejemplo 2 - Regresión de Poisson

```
In [38]: import numpy as np
from sklearn.linear_model import PoissonRegressor
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
```

Se dispone de un conjunto de datos con 214 observaciones diarias que contiene información meteorológica y el número de bicicletas registradas por día. Las variables consideradas son:

- Date: fecha de la observación.
- HIGH_T: temperatura máxima diaria.
- LOW_T: temperatura mínima diaria.
- PRECIP: cantidad de precipitación registrada.
- BB_COUNT: número de bicicletas registradas durante el día.

Debido a que la variable BB_COUNT representa un conteo, se desea construir un modelo de regresión de Poisson que permita analizar la relación entre las condiciones meteorológicas y la cantidad esperada de bicicletas registradas diariamente.

A partir de los datos proporcionados, realice lo siguiente:

- Explore el conjunto de datos, identificando las variables, sus tipos de datos y posibles valores atípicos mediante representaciones gráficas.
- Realice el preprocesamiento necesario, incluyendo la transformación de la variable Date y la creación de variables temporales como mes, día de la semana y día del mes.
- Defina la variable dependiente y las variables independientes, y divida los datos en conjuntos de entrenamiento y prueba.
- Ajuste un modelo de regresión de Poisson utilizando las variables meteorológicas y temporales seleccionadas como predictoras.
- Interprete los coeficientes obtenidos y evalúe la significancia estadística de las variables, analizando su efecto sobre el número esperado de bicicletas registradas.
- Evalúe el desempeño del modelo y compare los valores reales con los valores predichos mediante una métrica adecuada y representaciones gráficas.

```
In [41]: datos_bicicleta = pd.read_csv("nyc_bb_bicyclist_counts.csv")
```

```
In [43]: datos_bicicleta
```

Out[43]:

	Date	HIGH_T	LOW_T	PRECIP	BB_COUNT
0	1-Apr-17	46.0	37.0	0.00	606
1	2-Apr-17	62.1	41.0	0.00	2021
2	3-Apr-17	63.0	50.0	0.03	2470
3	4-Apr-17	51.1	46.0	1.18	723
4	5-Apr-17	63.0	46.0	0.00	2807
...	...	...	...	...	...
209	27-Oct-17	62.1	48.0	0.00	3150
210	28-Oct-17	68.0	55.9	0.00	2245
211	29-Oct-17	64.9	61.0	3.03	183
212	30-Oct-17	55.0	46.0	0.25	1428
213	31-Oct-17	54.0	44.0	0.00	2727

214 rows × 5 columns

In [45]: `datos_bicicleta.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 214 entries, 0 to 213
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        214 non-null    object
1   HIGH_T      214 non-null    float64
2   LOW_T       214 non-null    float64
3   PRECIP      214 non-null    float64
4   BB_COUNT    214 non-null    int64
dtypes: float64(3), int64(1), object(1)
memory usage: 8.5+ KB
```

In [47]: `# Ajustar el tipo de la variable Fecha`
`datos_bicicleta['Date'] = pd.to_datetime(datos_bicicleta['Date'], errors='coerce')`

C:\Users\USER\AppData\Local\Temp\ipykernel_8012\3926914016.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
datos_bicicleta['Date'] = pd.to_datetime(datos_bicicleta['Date'], errors='coerce')
```

In [49]: `datos_bicicleta`

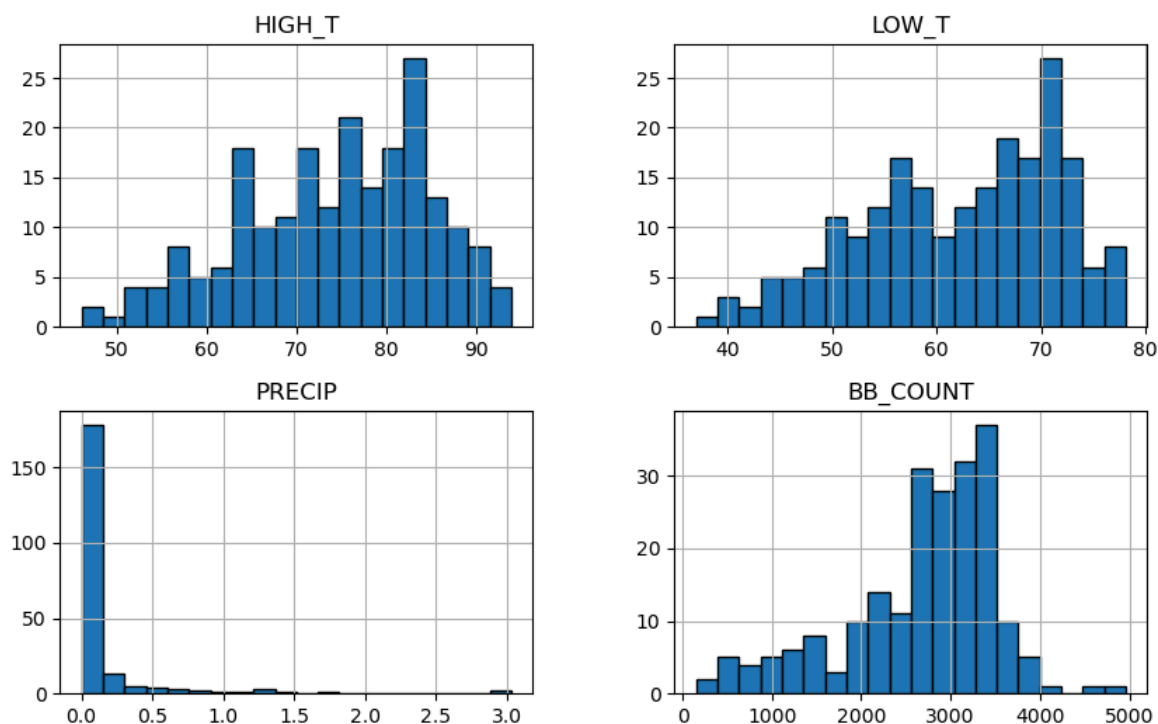
Out[49]:

	Date	HIGH_T	LOW_T	PRECIP	BB_COUNT
0	2017-04-01	46.0	37.0	0.00	606
1	2017-04-02	62.1	41.0	0.00	2021
2	2017-04-03	63.0	50.0	0.03	2470
3	2017-04-04	51.1	46.0	1.18	723
4	2017-04-05	63.0	46.0	0.00	2807
...	...	...	...	...	...
209	2017-10-27	62.1	48.0	0.00	3150
210	2017-10-28	68.0	55.9	0.00	2245
211	2017-10-29	64.9	61.0	3.03	183
212	2017-10-30	55.0	46.0	0.25	1428
213	2017-10-31	54.0	44.0	0.00	2727

214 rows × 5 columns

```
In [51]: plt.figure()
datos_bicicleta[["HIGH_T", "LOW_T", "PRECIP", "BB_COUNT"]].hist(figsize=(10,6))
plt.show()
```

<Figure size 640x480 with 0 Axes>

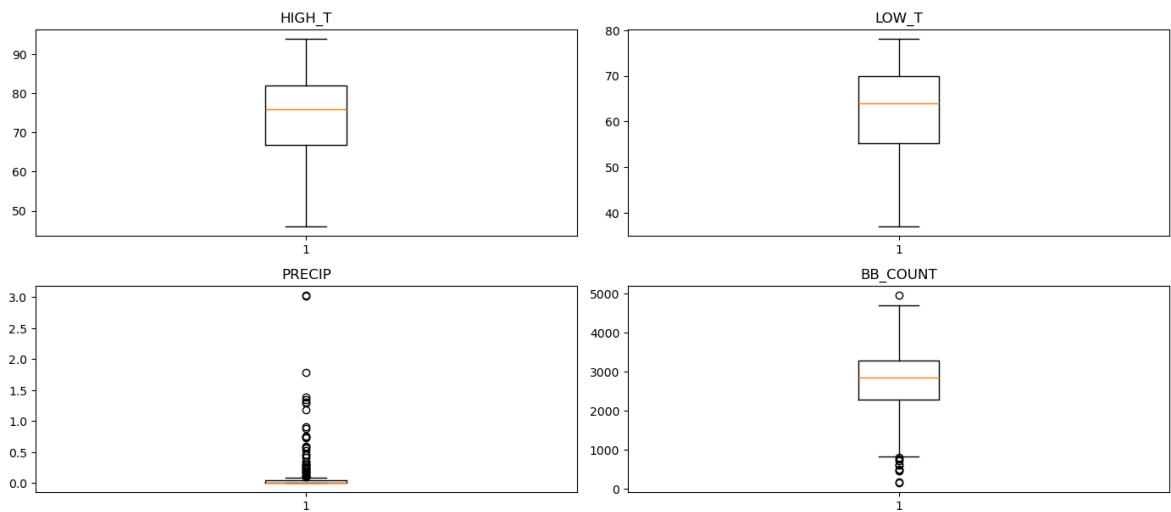


```
In [52]: vars = ["HIGH_T", "LOW_T", "PRECIP", "BB_COUNT"]
n = len(vars)

plt.figure(figsize=(14, 3*n))
for idx, var in enumerate(vars, 1):
    plt.subplot(n, 2, idx)
    plt.boxplot(datos_bicicleta[var])
```

```
plt.title(var)

plt.tight_layout()
plt.show()
```



A partir de los histogramas y diagramas de caja se observa que las temperaturas presentan una distribución relativamente concentrada, mientras que PRECIP presenta una fuerte asimetría hacia la derecha y numerosos valores extremos. En BB_COUNT también se identifican algunas observaciones alejadas del comportamiento central de los datos.

```
In [54]: datos_bicicleta["Mes"] = datos_bicicleta['Date'].dt.month
datos_bicicleta["Día semana"] = datos_bicicleta['Date'].dt.dayofweek
datos_bicicleta["Día"] = datos_bicicleta['Date'].dt.day
```

La variable Date fue transformada a un formato de fecha, lo que permitió extraer nueva información temporal. A partir de ella se generaron las variables mes, día de la semana y día del mes. Estas variables fueron incorporadas posteriormente al modelo con el objetivo de capturar posibles variaciones temporales en el número de bicicletas registradas.

```
In [55]: datos_bicicleta
```

Out[55]:

	Date	HIGH_T	LOW_T	PRECIP	BB_COUNT	Mes	Día semana	Día
0	2017-04-01	46.0	37.0	0.00	606	4	5	1
1	2017-04-02	62.1	41.0	0.00	2021	4	6	2
2	2017-04-03	63.0	50.0	0.03	2470	4	0	3
3	2017-04-04	51.1	46.0	1.18	723	4	1	4
4	2017-04-05	63.0	46.0	0.00	2807	4	2	5
...	...	...	...	...	...	...	...	...
209	2017-10-27	62.1	48.0	0.00	3150	10	4	27
210	2017-10-28	68.0	55.9	0.00	2245	10	5	28
211	2017-10-29	64.9	61.0	3.03	183	10	6	29
212	2017-10-30	55.0	46.0	0.25	1428	10	0	30
213	2017-10-31	54.0	44.0	0.00	2727	10	1	31

214 rows × 8 columns

```
In [59]: X_independientes = datos_bicicleta[["HIGH_T", "LOW_T", "PRECIP", "Mes", "Día sem
y_dependiente = datos_bicicleta["BB_COUNT"]
```

```
In [61]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_independientes, y_dependiente,
```

```
In [63]: poisson_reg = PoissonRegressor()
poisson_reg.fit(X_train, y_train)
```

```
Out[63]: ▼ PoissonRegressor ⓘ ?
PoissonRegressor()
```

```
In [65]: print("El intercepto de la regresión de Poisson es %s" %poisson_reg.intercept_)
```

El intercepto de la regresión de Poisson es 7.061001355317951

```
In [67]: #Mostrar coeficientes
coeficientes_poisson = pd.DataFrame({"Variable": X_independientes.columns, "Coef
print(coeficientes_poisson)
```

	Variable	Coeficiente
0	HIGH_T	0.024234
1	LOW_T	-0.014438
2	PRECIP	-0.789350
3	Mes	0.006372
4	Día semana	-0.024064
5	Día	0.000106

Los resultados indican que algunas variables presentan una relación positiva con el número esperado de bicicletas, mientras que otras presentan una relación negativa.

Destaca especialmente PRECIP, cuyo coeficiente negativo es considerablemente mayor en magnitud que el resto.

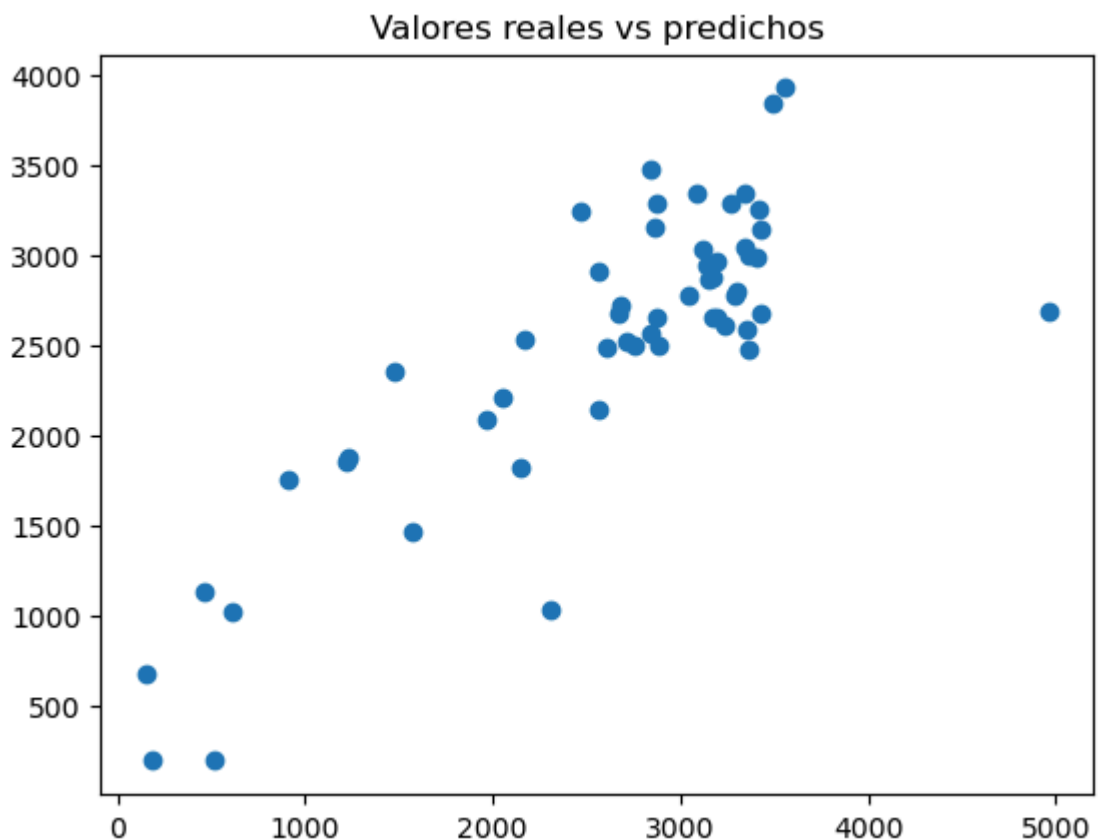
```
In [69]: print("El valor del D^2 es %s" %poisson_reg.score(X_train,y_train))
```

El valor del D^2 es 0.5285528175457943

Se logra explicar aproximadamente un 52.9 % de la desviación en comparación con un modelo que solo utiliza el intercepto. Por tanto, el modelo posee una capacidad explicativa moderada, aunque todavía existe una proporción importante de variabilidad que no está siendo capturada por las variables incluidas.

```
In [95]: y_pred = poisson_reg.predict(X_test)
```

```
plt.figure()
plt.title('Valores reales vs predichos')
plt.scatter(y_test,y_pred)
plt.show()
```



```
In [73]: import statsmodels.api as sm
X2 = sm.add_constant(X_train)
modelo_poisson = sm.GLM(y_train, X2, family=sm.families.Poisson())
resultados = modelo_poisson.fit()
```

```
In [75]: print(resultados.summary())
```

Generalized Linear Model Regression Results

=====						
Dep. Variable:	BB_COUNT		No. Observations:	160		
Model:	GLM		Df Residuals:	153		
Model Family:	Poisson		Df Model:	6		
Link Function:	Log		Scale:	1.0000		
Method:	IRLS		Log-Likelihood:	-11093.		
Date:	Sat, 15 Aug 2026		Deviance:	20638.		
Time:	08:51:41		Pearson chi2:	2.03e+04		
No. Iterations:	5		Pseudo R-squ. (CS):	1.000		
Covariance Type:	nonrobust					
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	7.0633	0.013	533.047	0.000	7.037	7.089
HIGH_T	0.0241	0.000	67.130	0.000	0.023	0.025
LOW_T	-0.0143	0.000	-37.406	0.000	-0.015	-0.014
PRECIP	-0.8000	0.009	-86.948	0.000	-0.818	-0.782
Mes	0.0063	0.001	7.901	0.000	0.005	0.008
Día semana	-0.0241	0.001	-31.603	0.000	-0.026	-0.023
Día	9.43e-05	0.000	0.547	0.584	-0.000	0.000
=====						

Según el modelo GLM, HIGH_T, LOW_T, PRECIP, Mes y Día semana presentan valores $p < 0.05$, por lo que muestran evidencia de una asociación estadísticamente significativa con el conteo de bicicletas. En cambio, la variable Día presenta $p = 0.584$, por lo que no resulta estadísticamente significativa en el modelo.

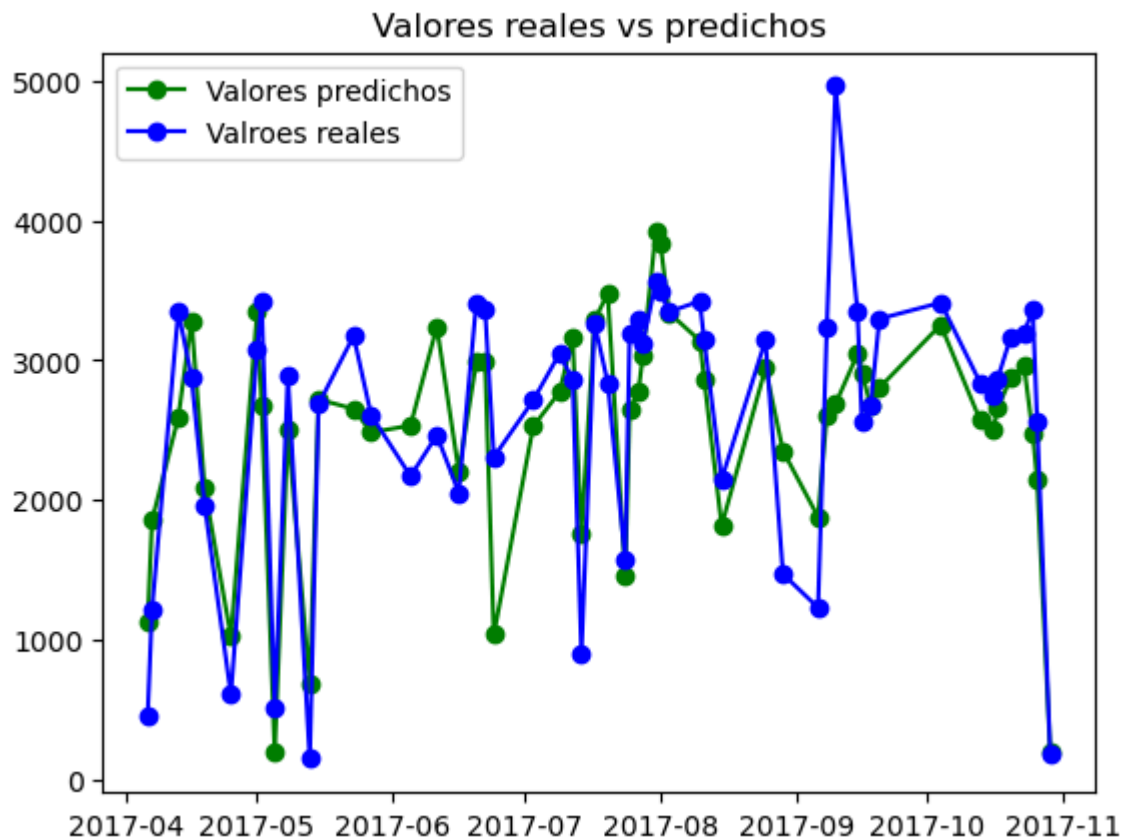
```
In [77]: y_pred = poisson_reg.predict(X_test)
indices = X_test.index
fechas_test = datos_bicicleta["Date"][indices]

orden = np.argsort(fechas_test)

fechas_ordenadas = fechas_test.iloc[orden]
y_pred_ordenado = y_pred[orden]
y_test_ordenado = y_test.iloc[orden]
```

```
In [91]: plt.title('Valores reales vs predichos')
plt.plot(fechas_ordenadas, y_pred_ordenado, 'go-', label='Valores predichos')
plt.plot(fechas_ordenadas, y_test_ordenado, 'bo-', label='Valroes reales')

plt.legend()
plt.show()
```



Gráficamente se observa que las predicciones siguen en términos generales el comportamiento de los valores observados, aunque existen días en los que las diferencias son considerables.